

Decompose

分块思想

简介

其实，分块是一种思想，而不是一种数据结构。

从 NOIP 到 NOI 到 IOI，各种难度的分块思想都有出现。

分块的基本思想是，通过对原数据的适当划分，并在划分后的每一个块上预处理部分信息，从而较一般的暴力算法取得更优的时间复杂度。

分块的时间复杂度主要取决于分块的块长，一般可以通过均值不等式求出某个问题下的最优块长，以及相应的时间复杂度。

分块是一种很灵活的思想，相较于树状数组和线段树，分块的优点是通用性更好，可以维护很多树状数组和线段树无法维护的信息。

当然，分块的缺点是渐进意义的复杂度，相较于线段树和树状数组不够好。

不过在大多数问题上，分块仍然是解决这些问题的一个不错选择。

下面是几个例子。

区间和

► 例题 [LibreOJ 6280 数列分块入门 4](#)

我们将序列按每 $\sqrt{n}$ 个元素一块进行分块，并记录每块的区间和。

最后一个块可能是不完整的（因为 $\sqrt{n}$ 很可能不是 n 的倍数），但是这对于我们的讨论来说并没有太大影响。

首先看查询操作：

- 若 l 和 r 在同一个块内，直接暴力求和即可，因为块长为 $\sqrt{n}$ ，因此最坏复杂度为 $O(\sqrt{n})$ 。
- 若 l 和 r 不在同一个块内，则答案由三部分组成：以 l 开头的不完整块，中间几个完整块，以 r 结尾的不完整块。对于不完整的块，仍然采用上面暴力计算的方法，对于完整块，则直接利用已经求出的 sum 求和即可。这种情况下，最坏复杂度为 $O(\sqrt{n})$ 。

接下来是修改操作：

- 若 l 和 r 在同一个块内，直接暴力修改即可，因为块长为 $\sqrt{n}$ ，因此最坏复杂度为 $O(\sqrt{n})$ 。
- 若 l 和 r 不在同一个块内，则需要修改三部分：以 l 开头的不完整块，中间几个完整块，以 r 结尾的不完整块。对于不完整的块，仍然是暴力修改每个元素的值（别忘了更新区间和），对于完整块，则直接修改 sum 即可。这种情况下，最坏复杂度和仍然为 $O(\sqrt{n})$ 。

利用均值不等式可知，当 $\sqrt{n}$ ，即 n 时，单次操作的时间复杂度最优，为 $O(\sqrt{n})$ 。

► 参考代码

区间和 2

上一个做法的复杂度是。

我们在这里介绍一种 的算法。

为了 询问，我们可以维护各种前缀和。

然而在有修改的情况下，不方便维护，只能维护单个块内的前缀和。

以及整块作为一个单位的前缀和。

每次修改。

询问：涉及三部分，每部分都可以直接通过前缀和得到，时间复杂度。

对询问分块

同样的问题，现在序列长度为，有 个操作。

如果操作数量比较少，我们可以把操作记下来，在询问的时候加上这些操作的影响。

假设最多记录 个操作，则修改，询问。

个操作之后，重新计算前缀和，。

总复杂度：。

时，总复杂度。

其他问题

分块思想也可以应用于其他整数相关问题：寻找零元素的数量、寻找第一个非零元素、计算满足某个性质的元素个数等等。

还有一些问题可以通过分块来解决，例如维护一组允许添加或删除数字的集合，检查一个数是否属于这个集合，以及查找第 大的数。要解决这个问题，必须将数字按递增顺序存储，并分割成多个块，每个块中包含 个数字。每次添加或删除一个数字时，必须通过在相邻块的边界移动数字来重新分块。

一种很有名的离线算法 [莫队算法](#)，也是基于分块思想实现的。

练习题

- [UVa - 12003 - Array Transformer](#)
- [UVa - 11990 Dynamic Inversion](#)
- [SPOJ - Give Away](#)
- [Codeforces - Till I Collapse](#)
- [Codeforces - Destiny](#)
- [Codeforces - Holes](#)
- [Codeforces - XOR and Favorite Number](#)
- [Codeforces - Powerful array](#)
- [SPOJ - DQUERY](#)

本页面主要译自博文 [Sqrt-декомпозиция](#) 与其英文翻译版 [Sqrt Decomposition](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[HeRaNO](#), [lr1d](#), [Xeonacid](#), [Chrogeek](#), [Enter-tainer](#), [gi-b716](#), [kenlig](#), [linghaiyi](#),
[StudyingFather](#), [YanJun-Zhao](#), [yiyangit](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用